LANGGRAPH MASTERY

Developing LLM Agents with LangGraph

ReAct Agents • LangGraph Graphs & State • Tools & Memory
Chatbot • Reflection • Reflexion Essay Writer • RAG Research Agent
LangSmith • Tavily AI • JWT Auth • Production Patterns

Production-Ready GenAI & Data Science Engineering

Prerequisites — Python

Before diving into LangGraph, ensure you are comfortable with the following Python concepts:

1. Flow control — if/else, loops, conditional logic.
2. Working with data structures — lists, dictionaries, tuples.
3. Functions — defining, calling, passing arguments, return values.
4. Type annotations — using Python's typing module for type hints (str, int, List, Dict, Optional).
5. Object-Oriented Programming (OOP) — classes, constructors (__init__), methods, inheritance.

Prerequisites — OpenAI & LangChain

6. Managing environment variables securely using .env files and python-dotenv.
7. Authenticating to LLM providers such as OpenAI and Anthropic using API keys.
8. Understanding the OpenAI Chat Completions API — roles (user, assistant, system), message format.
9. Working with LangChain prompt templates and chains.
10. Output parsers — extracting structured data from LLM responses.

SECTION 1 Theory — ReAct, LangGraph Concepts & Architecture

Chapter 1: ReAct — Synergizing Reasoning and Acting

ReAct (Reasoning + Acting) is a paradigm for building LLM-powered agents that interleave reasoning and action in a structured loop. It was introduced as a solution to two key limitations of language models used in isolation: models that only reason (but cannot take actions to retrieve information or affect the world) and models that only act (but do not think through their steps logically before acting).

ReAct agents solve complex, multi-step tasks by combining the best of both worlds: the language model reasons through what it knows and what it needs to find out, then takes an action (such as searching the web or querying a database), receives an observation from the environment, and loops back to reason again based on the new information.

1.1 The ReAct Loop

Phase	Description
Thought	The LM reasons about the current situation: what does it know? What is missing? What should it do next? This is the reasoning trace.
Action	The LM selects and executes a tool (e.g., Wikipedia search, calculator, API call). This is the action taken on the environment.
PAUSE	Execution pauses while the action is performed. The LM waits for the environment's response.
Observation	The result of the action is returned to the LM. This new information updates the LM's knowledge.
Repeat / Answer	The LM uses the observation to reason again. The loop continues until the LM has enough information to provide a final answer.

Why ReAct Over Standard Prompting?

A standard LLM prompt sends a question and gets an answer — but the model cannot look anything up, execute code, or check if its answer is correct. ReAct transforms the LLM into an autonomous agent that can reason about its own knowledge gaps, use tools to fill those gaps, and self-correct

based on real observations. This is the foundation of modern AI agent frameworks including LangChain agents and LangGraph.

Chapter 2: LangGraph — Core Concepts

LangGraph is an open-source framework built by the LangChain team, specifically designed for creating stateful, multi-agent AI systems. While LangChain provides building blocks (chains, agents, tools), LangGraph provides the scaffolding to orchestrate these components as graph-based workflows — giving developers fine-grained control over the flow of information, state, and decision-making in complex agent pipelines.

2.1 Core Terminologies

Term	Definition
Graph	The overall workflow structure. A directed graph where nodes represent units of work and edges define how data flows between them.
Nodes	Individual units of work — these are Python functions or LLM calls that receive the current state, perform computation, and return an updated state.
Edges (Normal)	Connect nodes in a fixed, unconditional sequence. Node A always goes to Node B.
Conditional Edges	Connect nodes via a routing function that decides which node to go to next based on the current state. Enables branching logic.
State	A shared data object (TypedDict) that flows through the graph. Every node reads from and writes to the state. LangGraph manages state transitions automatically.
State Machine	LangGraph builds state machines under the hood — it tracks the current state of the workflow and ensures tasks execute in the correct order.
Cyclic Graph	A graph where execution can loop back to an earlier node. This is fundamental for ReAct-style agents that need to iterate (reason → act → observe → reason...).
Human in the Loop (HITL)	A design pattern where human intervention is integrated into automated workflows at specific checkpoints for review, approval, or correction.

Persistence	The ability to save and restore the state of a workflow, enabling it to be paused and resumed across sessions.
Flow Engineering	A structured approach where complex problems are decomposed into a series of well-defined, iteratively refined steps — rather than a single monolithic prompt.

LangGraph vs. LangChain Agents

LangChain's standard AgentExecutor runs agents in a loop but offers limited control over the execution flow. LangGraph gives you explicit control: you define exactly which nodes run, in what order, and under what conditions. This makes LangGraph far better suited for production systems, multi-agent pipelines, and any workflow that requires conditional branching, loops, or state management.

SECTION 2

Practical — Projects with Code Explanations

Chapter 3: Project 1 — Building a ReAct Agent from Scratch

This project builds a fully functional ReAct agent without using any agent framework — pure Python and the OpenAI API. This hands-on approach is the best way to deeply understand how ReAct agents work before using higher-level abstractions like LangGraph.

Step 1 — Environment Setup

```
# Load environment variables from .env file
from dotenv import load_dotenv, find_dotenv

# find_dotenv() searches for .env file starting from current directory
# moving upward through parent directories until found
# load_dotenv() reads the key=value pairs and injects them into os.environ
# override=True ensures .env values override any existing environment variables
load_dotenv(find_dotenv(), override=True)
```

What this cell does: Environment variables (API keys, secrets) must never be hardcoded in source code — doing so is a severe security risk. The python-dotenv library solves this: you store sensitive values in a .env file (excluded from version control via .gitignore), and load_dotenv() makes them accessible via os.getenv() or os.environ. find_dotenv() reliably locates the .env file regardless of which directory the script is run from.

Step 2 — The Agent Class

```
from openai import OpenAI

client = OpenAI()   # Reads OPENAI_API_KEY from environment automatically
MODEL = 'gpt-4o-mini'

class Agent:
    def __init__(self, system=''):
        # Store the system prompt (sets the AI's persona/behaviour)
        self.system = system
        # Initialize empty conversation history
        self.messages = []
        # If a system prompt is provided, add it as the first message
        if self.system:
            self.messages.append({'role': 'system', 'content': system})

    def __call__(self, prompt):
        # Append user's message to conversation history
        self.messages.append({'role': 'user', 'content': prompt})
```

```
        # Get response from LLM
        result = self.execute()
        # Append LLM's response to maintain conversation context
        self.messages.append({'role': 'assistant', 'content': result})
        return result

    def execute(self, model=MODEL, temperature=0):
        # Send full conversation history to LLM and get response
        completion = client.chat.completions.create(
            model=model,
            temperature=temperature,    # 0 = deterministic output
            messages=self.messages      # full conversation context
        )
        # Extract and return the text content of the response
        return completion.choices[0].message.content
```

__init__(self, system=''): The constructor initializes the agent. self.messages is the conversation history — this is a list of message dictionaries, each with 'role' (system/user/assistant) and 'content' keys. The system message is placed first, before any user messages. It sets the agent's persona, capabilities, and output format instructions.

__call__(self, prompt): Making the class callable (via __call__) allows instances to be used like functions: agent('what is Python?'). This method appends the user message, calls execute() to get the LLM response, then appends the assistant response. By maintaining self.messages across calls, the agent has persistent memory of the full conversation.

execute(self, model, temperature): This method is the bridge to the OpenAI API. temperature=0 produces deterministic, reproducible outputs (the same input always produces the same output) — important for ReAct agents where consistent structured output format is required. The full self.messages list is sent each time, giving the LLM complete conversation context.

Step 3 — The ReAct Prompt

The ReAct prompt is the most critical component. It instructs the LLM to follow the Thought → Action → PAUSE → Observation loop and defines exactly what tools are available and how to call them. An example prompt structure:

```
react_prompt = '''
You run in a loop of Thought, Action, PAUSE, Observation.
At the end of the loop you output an Answer.

Use Thought to describe your reasoning about the question you have been asked.
Use Action to run one of the available actions - then return PAUSE.
Observation will be the result of running those actions.

Available actions:
wikipedia:
    Runs a search on Wikipedia and returns a relevant snippet.
    Example: wikipedia: LangGraph framework

Example session:
Question: What is the capital of France?
Thought: I should look up France on Wikipedia.
Action: wikipedia: France
PAUSE
```

```
Observation: France is a country in Western Europe. Its capital is Paris.
Thought: I now know the capital of France.
Answer: The capital of France is Paris.
'''.strip()
```

What this code does: The ReAct prompt is a detailed instruction set that fundamentally changes how the LLM responds. Instead of giving a direct answer, it follows the structured Thought/Action/PAUSE loop. The PAUSE token is critical — it signals the code to stop, execute the called action (e.g., actually search Wikipedia), and then provide the result back to the LLM as an Observation. Without the PAUSE mechanism, the LLM would hallucinate results rather than actually calling tools.

Step 4 — Creating a Tool (Wikipedia Search)

```
import httpx

def wikipedia(q: str) -> str:
    '''Search Wikipedia and return the snippet of the top result.'''
    response = httpx.get(
        'https://en.wikipedia.org/w/api.php',
        params={
            'action': 'query',     # Perform a query operation
            'list': 'search',      # Type of query: text search
            'srsearch': q,         # The search term
            'format': 'json'       # Return results as JSON
        }
    )

    # Parse JSON response and navigate to search results
    results = response.json().get('query').get('search', [])

    # Return None if no results found
    if not results:
        return None

    # Return the snippet from the top search result
    return results[0]['snippet']

# Tool registry: maps action names to Python functions
known_actions = { 'wikipedia': wikipedia }
```

What this code does: This is a 'tool' in the ReAct architecture — a Python function the agent can call to interact with the external world. httpx.get() makes an HTTP GET request to Wikipedia's public API. The params dictionary is URL-encoded automatically. The response comes back as JSON; we navigate the nested structure with .get() to safely extract the search results list. results[0]['snippet'] returns the text excerpt from the top match. The known_actions dictionary is the agent's tool registry — it maps action name strings (as written in the LLM's output) to actual Python functions.

Step 5 — Testing & Automating the Agent

```
import re
```

```
# Initialize agent with ReAct prompt
abot = Agent(react_prompt)

# Regex pattern to detect Action calls in LLM output
# Matches: 'Action: tool_name: query'
action_re = re.compile(r'^Action: (\w+): (.*)$', re.MULTILINE)

def query(question: str, max_turns: int = 5):
    '''Run the full ReAct loop for a given question.'''
    i = 0
    next_prompt = question

    while i < max_turns:
        i += 1
        # Send prompt, get LLM response
        result = abot(next_prompt)
        print(f'--- Turn {i} ---')
        print(result)

        # Check if LLM wants to take an action
        actions = action_re.findall(result)

        if not actions:
            # No action requested — LLM has final answer
            print('Final Answer reached.')
            return

        # Execute the requested action
        action_name, action_input = actions[0]
        if action_name not in known_actions:
            raise ValueError(f'Unknown action: {action_name}')

        # Call the tool and get observation
        observation = known_actions[action_name](action_input)
        print(f'Observation: {observation}')

        # Feed observation back to agent for next reasoning step
        next_prompt = f'Observation: {observation}'

# Test the agent
query('What programming language is LangGraph written in?')
```

What this code does: The query() function orchestrates the full ReAct loop. action_re is a compiled regex that parses the LLM's structured output to extract the tool name and its input. The while loop implements the Thought → Action → PAUSE → Observation cycle: after each LLM response, we check whether it contains an Action call; if it does, we execute the corresponding Python function, collect the observation, and feed it back as the next prompt. If no action is detected, the LLM has reached its final answer and the loop terminates. max_turns prevents infinite loops in case the agent gets stuck.

Chapter 4: Project 2 — LangGraph Chatbot

This project builds a production-quality chatbot using LangGraph's StateGraph, with progressively added features: basic conversation → web search tool → persistent memory.

Step 1 — Imports & Environment Setup

```
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from typing import Annotated
from typing_extensions import TypedDict
from langchain_openai import ChatOpenAI
from langchain_groq import ChatGroq
from dotenv import load_dotenv, find_dotenv
import os

load_dotenv(find_dotenv(), override=True)
```

What this cell does: StateGraph is the core LangGraph class — it defines the graph structure (nodes, edges) and manages state transitions. add_messages is a special reducer function for the messages field: instead of replacing the messages list on each state update, it appends new messages to the existing list. This is what gives LangGraph chatbots persistent conversation memory within a session. TypedDict is used to define the State schema with full type safety.

Step 2 — Defining State, Chatbot Node & Building the Graph

```
# Step 1: Setup LLM (Groq is faster and free-tier friendly)
llm = ChatGroq(
    groq_api_key=os.getenv('GROQ_API_KEY'),
    model='llama3-8b-8192'
)

# Step 2: Define the graph's State schema
# Annotated[list, add_messages] tells LangGraph to use add_messages
# reducer — new messages are APPENDED, not replaced
class State(TypedDict):
    messages: Annotated[list, add_messages]

# Step 3: Define the chatbot node function
# Receives current state, calls LLM, returns updated state
def chatbot(state: State) -> State:
    response = llm.invoke(state['messages'])
    return {'messages': [response]}

# Step 4: Build and compile the graph
graph = StateGraph(State)
graph.add_node('chatbot', chatbot)    # Register the chatbot node
graph.add_edge(START, 'chatbot')      # Entry point -> chatbot
graph.add_edge('chatbot', END)         # chatbot -> Exit
app = graph.compile()                  # Compile to executable graph

# Step 5: Gradio chat interface
```

```
import gradio as gr
from typing import List, Dict

def chat(user_input: str, history: List[Dict[str, str]]):
    messages = history + [{'role': 'user', 'content': user_input}]
    result = app.invoke({'messages': messages})
    return result['messages'][-1].content

# Step 6: Launch
gr.ChatInterface(chat, type='messages').launch()
```

What this code does — State: The State TypedDict defines the data structure shared across all nodes. The Annotated[list, add_messages] annotation does two things: declares the type as a list, and specifies add_messages as the reducer function. A reducer determines how state updates are merged — add_messages appends new messages to the existing list rather than overwriting it.

What this code does — chatbot node: A LangGraph node is simply a Python function that takes the current State dictionary and returns a partial state update. llm.invoke(state['messages']) sends the full conversation history to the LLM. The return value {'messages': [response]} triggers the add_messages reducer to append the new response.

What this code does — graph compilation: StateGraph.compile() validates the graph structure (checks for unreachable nodes, missing edges, etc.) and creates an executable application object (app). START and END are special LangGraph constants representing the graph's entry and exit points. graph.compile() returns a CompiledStateGraph that supports invoke(), stream(), and ainvoke() methods.

Step 3 — Adding Web Search with Tavily AI

Tavily AI is a search engine purpose-built for LLM integration. Unlike general web search APIs, Tavily returns AI-optimized search results — clean, factual text snippets rather than raw HTML — making it ideal for feeding into language models.

```
from langchain_community.tools.tavily_search import TavilySearchResults
import os

# Initialize Tavily search tool (max_results controls response length)
tool = TavilySearchResults(max_results=3)  # Returns top 3 search results
tools = [tool]

# Bind tools to LLM — the model can now decide to call these tools
llm_with_tools = llm.bind_tools(tools)

# Agentic Search concept:
# Standard search: user types query → search engine returns page links
# Agentic search: AI agent decides WHAT to search, WHEN to search,
# and HOW to use the results — all as part of autonomous reasoning.

# Example: direct Tavily query
from tavily import TavilyClient
client = TavilyClient(api_key=os.environ.get('TAVILY_API_KEY'))
response = client.search(query='What is the Bitcoin price today?')
print(response)
```

What this code does: TavilySearchResults is a pre-built LangChain tool wrapper around Tavily's API. llm.bind_tools(tools) is a crucial step — it tells the LLM what tools are available and how to call them using OpenAI's function calling / tool use API. When the LLM determines that external information is needed, it returns a structured tool call (rather than a text response), which LangGraph's ToolNode then executes automatically.

Step 4 — Adding Persistent Memory with Checkpointing

```
from langgraph.checkpoint.memory import MemorySaver
from langgraph.checkpoint.sqlite import SqliteSaver

# In-memory checkpointer (lost when program restarts)
memory = MemorySaver()

# SQLite checkpointer (persists to disk — survives restarts)
sqlite_memory = SqliteSaver.from_conn_string(':memory:')

# Compile graph with checkpointer for persistence
app = graph.compile(checkpointer=memory)

# Each conversation session needs a unique thread_id
# Different thread_ids = completely separate conversation histories
config_user_1 = {'configurable': {'thread_id': 'user_001'}}
config_user_2 = {'configurable': {'thread_id': 'user_002'}}

# Stream response with memory
prompt = 'Hi! My name is Arjun and I am a data scientist.'
events = app.stream(
    {'messages': [('user', prompt)]},
    config_user_1,
    stream_mode='values'  # stream full state after each node
)
for event in events:
    event['messages'][-1].pretty_print()
```

What this code does: Checkpointing is what transforms a stateless graph into a stateful, memory-aware application. Without a checkpointer, every call to app.invoke() starts with a blank state — the agent has no memory of previous turns. With a checkpointer, LangGraph saves the complete state after every node execution. The thread_id is the key that identifies which conversation history to load/save — each unique thread_id represents a separate, isolated conversation session. This is how you support multiple concurrent users, each with their own private memory.

Chapter 5: Project 3 — Reflection Tweet Generator

This project introduces the Reflection pattern — a powerful technique where the LLM evaluates and improves its own outputs through iterative critique. Instead of accepting the first response, the system runs multiple Generate → Reflect cycles until the output reaches acceptable quality.

5.1 What is Reflection in LLMs?

Reflection is the process of prompting an LLM to look back at its own previous output, assess its quality, identify weaknesses, and suggest improvements. The three core operations are:

Operation	Description
Monitor	Continuously observe the actions taken and outputs produced by the agent.
Evaluate	Critically assess the effectiveness and quality of those outputs against defined criteria.
Adapt	Modify the generation strategy based on the evaluation to improve future outputs.

In a LangGraph Reflection agent, the graph has two nodes: a Generate node (produces content) and a Reflect node (critiques the content). A conditional edge between them determines whether to loop back for another generation cycle or exit with the final answer. This mirrors the human creative process: draft → review → revise → repeat.

Why Reflection Outperforms Single-Shot Prompting

A single-shot prompt asks the LLM to get it right on the first try. Reflection acknowledges that first drafts are rarely optimal and builds improvement into the pipeline. Studies show reflection-based systems significantly outperform one-shot systems on complex writing, coding, and reasoning tasks — with quality improving measurably over 2-4 reflection cycles.

Chapter 6: Project 3.5 — Reflexion Essay Writer

The Reflexion Essay Writer is a more sophisticated multi-node agent that combines planning, external research, generation, and critique in a full iterative writing pipeline.

6.1 Graph Architecture

Node	Responsibility
__start__	Entry point. Receives the essay topic from the user.

planner	Plans the essay structure — outlines key arguments, sections, and research directions.
research_plan	Executes web searches based on the plan to retrieve relevant documents and evidence.
generate	Writes the essay (or revision) based on the plan and research. This is the core content generation node.
reflect	Critiques the generated essay — identifies weaknesses, gaps, factual errors, and style issues.
research_critique	Performs additional research specifically targeting the weaknesses identified by the reflect node.
__end__	Exit point. Returns the final essay after the quality threshold is met.

The conditional edge after the generate node is the heart of this system: if the essay has been revised fewer than N times, route to reflect → research_critique → generate again. If the iteration limit is reached or quality is sufficient, route to __end__. This creates a self-improving loop that produces progressively better essays with each cycle.

Chapter 7: LangSmith — Observability & Debugging

LangSmith is a unified developer platform for debugging, testing, evaluating, and monitoring LLM applications. It integrates seamlessly with LangChain and LangGraph but can also function independently with any LLM provider.

Feature	Description
Tracing & Debugging	Provides detailed trace logs of every LLM call, tool invocation, and node execution. Groups operations into 'traces' — the entire execution triggered by one user request. Essential for understanding agent behaviour.
Evaluation & Testing	Allows creation of datasets with expected inputs and outputs. Run automated evaluations to measure whether your LLM application produces correct results across diverse test cases.
Monitoring & Observability	Real-time tracking of key metrics: latency, cost per request, token usage, error rates. Quickly identifies performance regressions or anomalies.
Collaboration & Feedback	Annotation queues and feedback collection tools allow teams to add human labels and insights to traces — improving evaluation datasets and model fine-tuning.
Versioning & Comparison	Side-by-side comparison of different prompt versions, model versions, or agent configurations — making it easy to measure the impact of changes.

LangSmith Setup

Enable LangSmith tracing by setting two environment variables: LANGCHAIN_API_KEY (your LangSmith API key) and LANGCHAIN_TRACING_V2=true. With these set, every LangChain/LangGraph call is automatically traced and visible in the LangSmith dashboard — no code changes required.

Chapter 8: Final Project — RAG Research Agent

The final project combines all previously learned components into a production-grade, autonomous research agent. Given a research topic, it automatically fetches academic papers from ArXiv, processes them into a searchable vector database, retrieves relevant information using RAG, and synthesizes a comprehensive research report.

8.1 System Architecture

Component	Technology & Role
Paper Discovery	ArXiv API — fetches metadata for recent research papers on the topic into a Pandas DataFrame.
Document Download	Downloads the full PDF files of selected papers programmatically.
Text Extraction & Chunking	Splits PDFs into overlapping text chunks for embedding. LangChain text splitters handle this.
Vector Database	Pinecone — stores vector embeddings of all text chunks for semantic similarity search.
RAG Search Tool	Retrieves the most relevant text chunks from Pinecone given a research query (rag_search node).
Web Search Tool	Tavily AI — supplements RAG with real-time web search when the vector database lacks coverage (web_search node).
ArXiv Fetch Tool	Directly fetches specific papers from ArXiv by ID (fetch_arxiv node).
Oracle (Decision Maker)	The central LLM node that decides which tool to call next based on the research question and current state.
Final Answer	Synthesizes all collected information into a structured, citation-rich research report.

8.2 Graph Flow

11. Start: User provides a research topic.
12. Oracle node: LLM analyzes the question and decides which tool to use first (rag_search_filter, rag_search, fetch_arxiv, or web_search).
13. Tool execution: The selected tool runs, and its output is added to the agent state.
14. Loop: Oracle receives the tool output as an observation, reasons about what is still needed, and either calls another tool or decides the research is complete.

15. final_answer node: When Oracle has sufficient information, it routes to final_answer, which synthesizes everything into a polished research report.
16. End: The final report is returned to the user.

End of LangGraph Mastery — Complete Reference Guide